

Laboratório 1 Introdução ao *Assembly***Linguagem *Assembly* do Microprocessador RISC-V**

A linguagem *assembly*, ou linguagem de montagem, é uma representação dos comandos executados por um microprocessador. Uma instrução de *assembly* clássico possui dois componentes:

	<code>add</code>	<code>s1, s2, s3</code>
<i>mnemônico (nome para lembrar o propósito)</i>		<i>operandos (registradores, constantes, memória)</i>

Um registrador é formado por flip-flops em paralelo e é usado como uma variável, guardando um valor numérico de 32 bits no nosso caso. Inicialmente, usaremos apenas os registradores s0 a s11.

O utilitário que transforma um programa em código *assembly* para o código de máquina (digamos o “executável”, o binário) se chama *montador (assembler)*, e não compilador.

Instrução de Soma

Para o RISC-V, a instrução básica é a soma dos valores vindos de dois registradores, com o resultado armazenado em um terceiro. Sintaxe: `add <destino>, <fonte1>, <fonte2>`. Por exemplo:

```
add s3, s7, s2      # realiza s3 ← s7+s2
```

Note-se que praticamente todos os montadores não diferenciam maiúsculas de minúsculas, porém podem ser bastante chatos com espaços e tabulações (um espaço depois ou antes de uma vírgula pode fazer um montador não reconhecer o comando). Tudo após o # é comentário.

Carga de Constantes

A instrução `addi <destino>, <fonte>, <constante>` irá somar o valor do registrador fonte à constante e armazenar no registrador destino. Já que o registrador zero é fixo e possui a constante zero armazenada nele (e é não modificável), podemos usar algo como:

```
addi s0, zero, 345  # carrega a constante 345 em s0
```

As constantes podem ser negativas. Note ainda que a constante obrigatoriamente deve ser o terceiro operando, não o segundo.

Montador-Simulador Integrado

O RARS IDE é bastante amigável e em Java¹, mas há alternativas². Rodem o bicho, programem e simulem o seguinte:



Carregue as constantes 123, 456 e 789 nos registradores s0, s1 e s2. Some tudo e coloquem o resultado em s3. Confirme na janela de registradores o resultado.

Nota: sempre dê um ou dois TABs no início de cada linha, fazendo um recuo para os comandos. Alguns montadores antigos *exigem* isso, e é boa prática de programação *assembly*. Ex.:

```
→      →   addi s0, zero, 43
→      →   add  s2, s0, zero
```

1 A última versão estável se encontra em <https://github.com/TheThirdOne/rars/releases/latest>, é baixar e usar direto. Basta googar “rars riscv” se precisar achar.

2 Por exemplo, tem o Venus RISC-V simulator, que roda online em <https://venus.cs61c.org/> ou que pode ser usado como plugin dentro do VSCode.

- Calcule $30 \cdot 1234$ com o mínimo de instruções que você conseguir. Confirme na janela de registradores o resultado. Não use instruções de multiplicação nem de *branch*.

Saltos Condicionais

A execução das instruções é, a princípio, sequencial. Para quebrarmos a sequência, podemos usar um comando de salto condicional (*branch*), útil para implementar *ifs*, *elses* e *whiles*.

Inicialmente, usaremos apenas `bne <reg1>, <reg2>, <destino>`, bastante simples: se `<reg1>` é diferente de `<reg2>`, o programa é desviado para `<destino>`; caso contrário, prossegue na execução normal para a próxima instrução. O `bne` significa *branch if not equal*.

O destino é dado por um *label* (rótulo) que identifica a linha-alvo. O *label* usualmente começa na primeira coluna e é seguido por dois pontos. No exemplo bem besta e inútil a seguir, o *label* chama-se “desvio:”

```
                addi s0, zero, 43
                add  s2, s0, zero
                bne  s2, zero, desvio
                add  s0, s0, s0      # não será executada
desvio:         addi s2, s2, -1     # vai repetir 43 vezes
                bne  s2, zero, desvio
                add  s5, zero, s0   # s5 termina com 43
```

- Faça um *loop* de 30 vezes repetindo a soma de 23 em um registrador. Ao final, o valor $30 \cdot 23$ deverá ser carregado em s5.

- Calcule a soma dos 40 primeiros números ímpares e a coloque no registrador s1.

Exercícios (Entregar um deles no Moodle)

- Calcule $\sum i^2$ para *i* de 1 a 20. Resultado no registrador s3. Rode e confira. Não use instruções de multiplicação.

- Faça um programa que calcula o fatorial de 8.

Fixação

Quais das instruções abaixo existem e quais não existem no RISC-V, segundo este PDF? Considere que `dif_zero` e `nao_chegou` são apenas *labels* para outras instruções.

1. `add s5, zero, zero`
2. `addi s5, zero, s0`
3. `bne s5, 33, nao_chegou`
4. `bne s5, s2, nao_chegou`
5. `add s5, s5, 1`
6. `add s5, zero, -14`
7. `add s5, 0, -14`
8. `addi s5, s5, 1`
9. `bne s5, s2, nao_chegou`
10. `addi s5, zero, -14`
11. `add s5, 3, s3`
12. `bne s5, 0, dif_zero`
13. `add s5, zero, 0`
14. `bne s5, zero, s0`